



CAR RENTAL FLEET MANAGER

Programming for Problem Solving (PSPJ) – Java

Presented by: Gundu Meghana

BTech CSE – 1st Year



ABSTRACT

Project Overview

The **Car Rental Fleet Manager** is a console-based Java application designed to manage a small fleet of rental cars. It enables users to:

- View and search for available vehicles
- Rent and return cars with cost calculation
- Organize fleet information with sorting

The project demonstrates fundamental Java programming concepts from variables, operators, control structures, arrays, methods, searching, and sorting – with no advanced APIs or OOP concepts.

Problem Identification & Objectives

The Problem

Manual fleet management makes it difficult to track availability, rental prices, and customer selections efficiently – leading to errors and delays.

Objectives

- Maintain car information and track availability
- Search, rent, and return cars efficiently
- Calculate rental costs and sort fleet by price
- Demonstrate Java PSP Modules 1, 2, 3 concepts

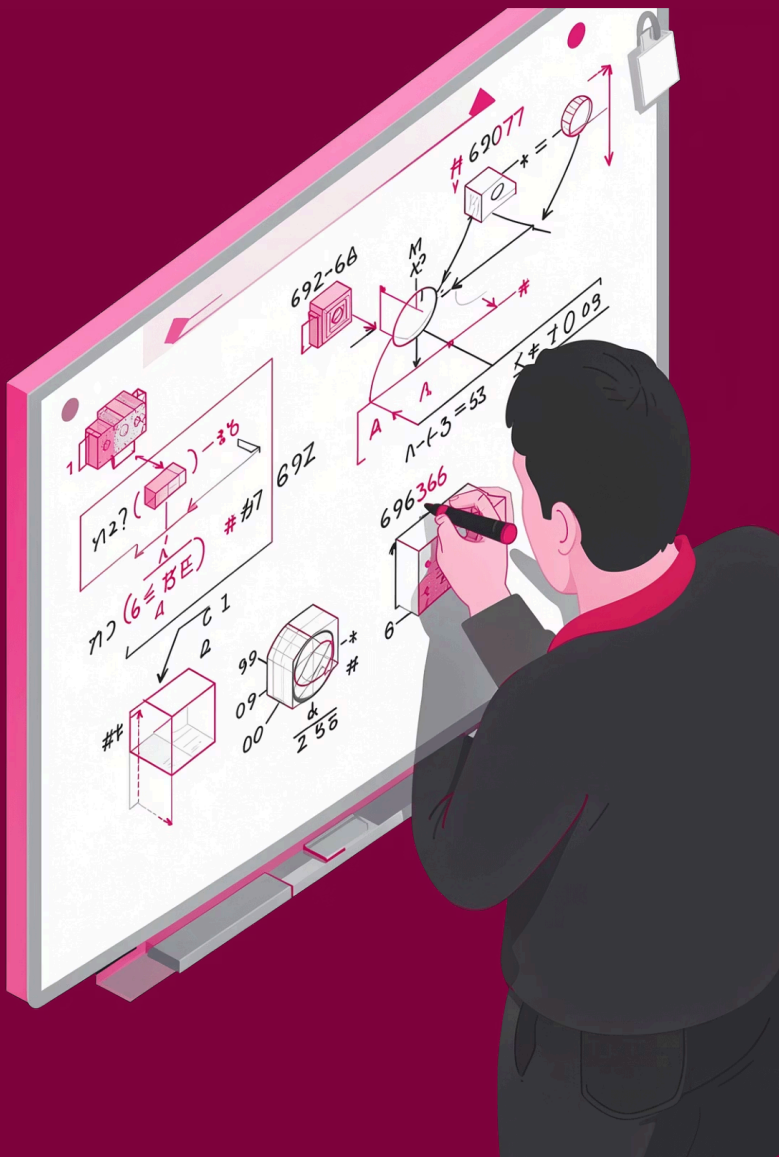
Advantages & Disadvantages

Advantages

- Simple, menu-driven interface for easy navigation
- Efficient parallel array structure for fleet management
- Fast linear search and sorting operations
- No external dependencies or complex frameworks
- Demonstrates core Java fundamentals clearly

Disadvantages

- Limited to 5 cars (fixed array size)
- No persistent data storage (data lost on exit)
- Console-based interface lacks visual appeal
- No user authentication or security features
- Cannot handle concurrent bookings or real-time updates



Module-Wise Concept Mapping

Fundamentals

- Variables & data types
- Arithmetic & relational operators
- Scanner for input
- Formatted output (printf)
- Ternary operator

Control Flow

- if-else decision making
- switch statement
- for, while, do-while loops
- break and continue
- Nested loops for sorting

Modular Programming

- Methods with parameters
- Return values
- Method overloading
- Arrays & parallel arrays
- Linear search & bubble sort

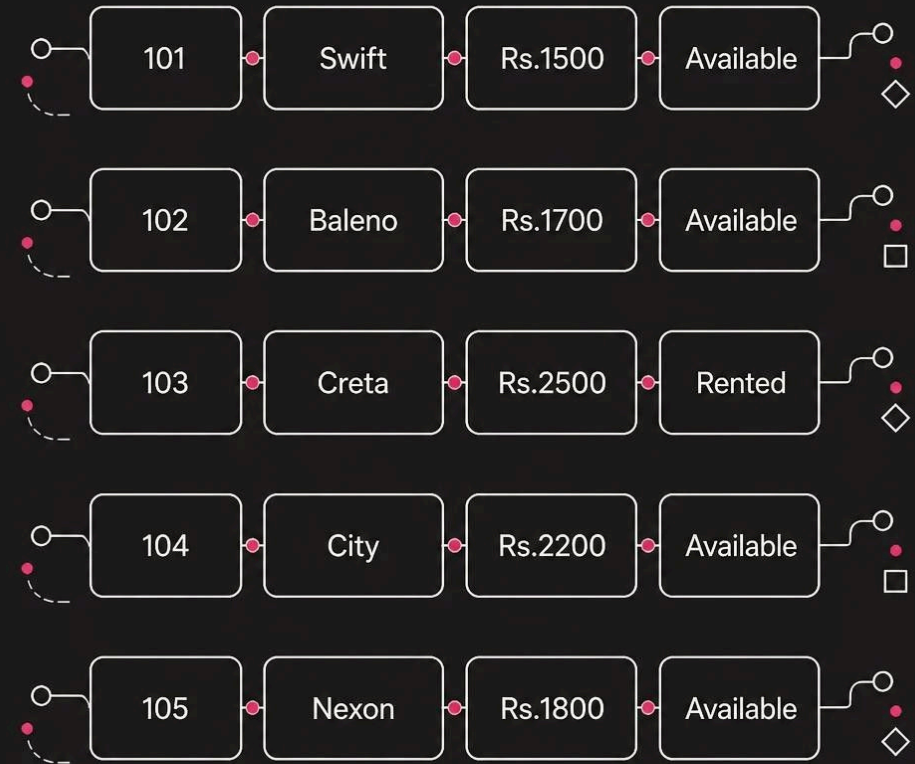
System Design & Data Structure

Parallel Array Architecture

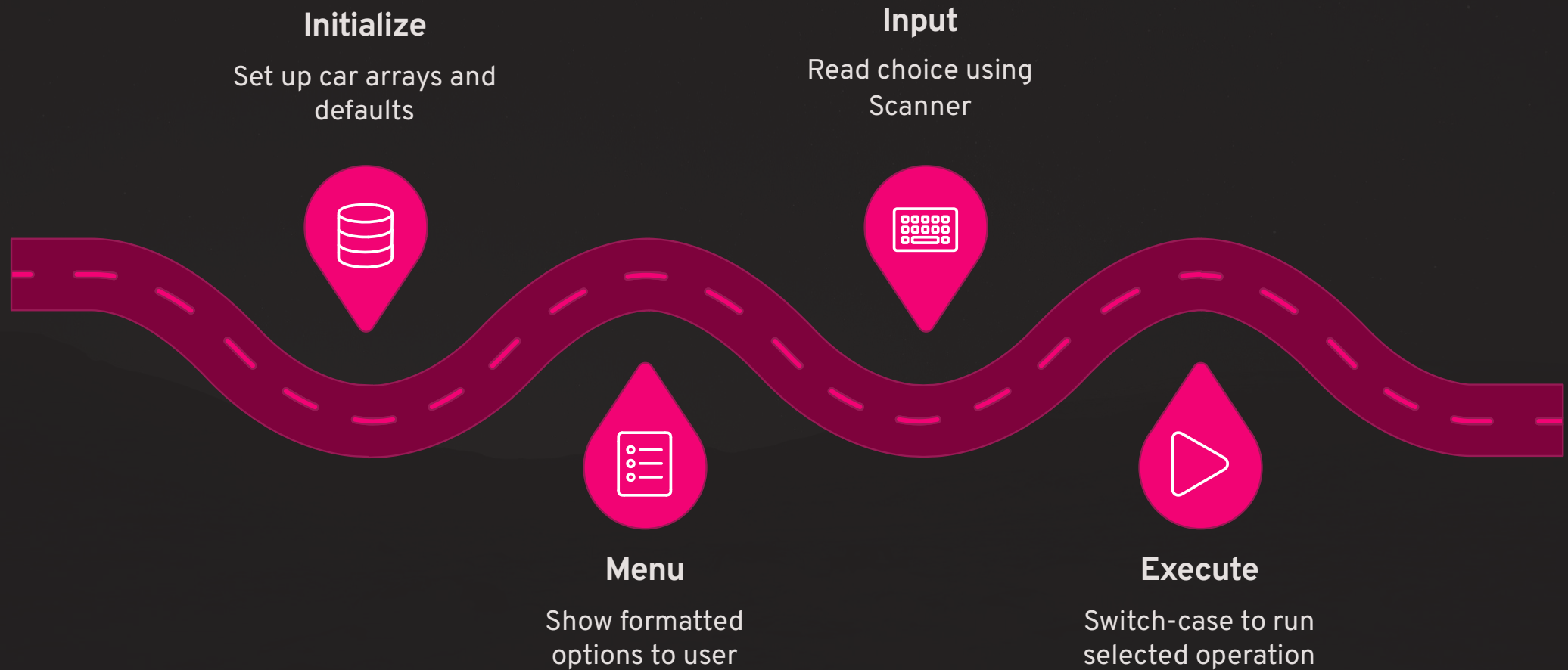
Each array index represents one car. Five parallel arrays store all fleet data efficiently – no classes or objects required.

```
carId[]      = {101, 102, 103, 104, 105}
carName[]    = {"Swift","Baleno","Creta",
               "City","Nexon"}
pricePerDay[] = {1500, 1700, 2500, 2200, 1800}
available[]   = {true, true, false, true, true}
fleetSize    = 5
```

📌 Index 0 → Swift (101), Index 1 → Baleno (102), etc.
All arrays stay in sync.



Algorithm & Flowchart



The application follows a **do-while driven menu loop**: initialize → display → read → switch → operate → display → repeat until choice = 8 (Exit).

Implementation: Key Methods

displayCars()

Loops through all arrays using a **for loop**, prints a formatted table with printf. Ternary operator used for status display. *(Module 1 + 2 + 3)*

searchCar(int id) & searchCar(String name)

Method overloading – two versions of searchCar perform linear search by ID or by name using equalsIgnoreCase(). Returns index or -1. *(Module 3)*

rentCar(Scanner sc) & returnCar(Scanner sc)

Uses if-else chains to validate availability, updates the available[] array, and calls calculateRent() for cost. *(Module 2 + 3)*

sortCarsByPrice()

Bubble sort using nested for loops on cloned parallel arrays. Swaps all four arrays together to maintain data integrity. *(Module 2 + 3)*



Complete Java Implementation

```
import java.util.Scanner;

public class CarRentalFleetManager {

    // MODULE 1: Arrays & Variables
    static int[] carId = {101, 102, 103, 104, 105};
    static String[] carName = {"Swift","Baleno","Creta","City","Nexon"};
    static int[] pricePerDay = {1500, 1700, 2500, 2200, 1800};
    static boolean[] available = {true, true, false, true, true};
    static int fleetSize = 5;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;
        // MODULE 2: do-while loop
        do {
            displayMenu();
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            // MODULE 2: switch statement
            switch(choice) {
                case 1: displayCars(); break;
                case 2: searchCarMenu(sc); break;
                case 3: rentCar(sc); break;
                case 4: returnCar(sc); break;
                case 5: calculateRentMenu(sc); break;
                case 6: sortCarsByPrice(); break;
                case 7: displaySummary(); break;
                case 8: System.out.println("Thank you!"); break;
                default: System.out.println("Invalid choice!");
            }
            while(choice != 8);
            sc.close();
        }

        // MODULE 1: Formatted Output
        static void displayMenu() {
            System.out.println("\n===== CAR RENTAL FLEET MANAGER =====");
            System.out.println("1. Display Cars 5. Calculate Rent");
            System.out.println("2. Search Car 6. Sort by Price");
            System.out.println("3. Rent Car 7. Fleet Summary");
            System.out.println("4. Return Car 8. Exit");
            System.out.println("=====");
        }

        // MODULE 3: Method with for loop & arrays
        static void displayCars() {
            System.out.printf("\n%-6s %-12s %-12s %s\n","ID","Model","Price/Day","Status");
            System.out.println("-----");
            for(int i = 0; i < fleetSize; i++) {
                String status = available[i] ? "Available" : "Rented";
                System.out.printf("%-6d %-12s Rs.%-9d %s\n",
                    carId[i], carName[i], pricePerDay[i], status);
            }
        }

        // MODULE 3: Method Overloading – Search by ID
        static int searchCar(int id) {
            for(int i = 0; i < fleetSize; i++)
                if(carId[i] == id) return i;
            return -1;
        }

        // MODULE 3: Method Overloading – Search by Name
        static int searchCar(String name) {
            for(int i = 0; i < fleetSize; i++)
                if(carName[i].equalsIgnoreCase(name)) return i;
            return -1;
        }

        // MODULE 3: Method with return value
        static int calculateRent(int days, int price) {
            return days * price; // MODULE 1: Arithmetic operator
        }

        // MODULE 3: Method with parameters and if-else
        static void rentCar(Scanner sc) {
            System.out.print("Enter Car ID to rent: ");
            int idx = searchCar(sc.nextInt());
            if(idx == -1) { System.out.println("Car not found!"); }
            else if(!available[idx]) { System.out.println("Already rented!"); }
            else {
                System.out.print("Rental days: ");
                int days = sc.nextInt();
                if(days > 0) {
                    available[idx] = false;
                    System.out.printf("Rented: %s | Days: %d | Total: Rs.%d\n",
                        carName[idx], days, calculateRent(days, pricePerDay[idx]));
                } else { System.out.println("Invalid number of days!"); }
            }
        }

        static void returnCar(Scanner sc) {
            System.out.print("Enter Car ID to return: ");
            int idx = searchCar(sc.nextInt());
            if(idx == -1) { System.out.println("Car not found!"); }
            else if(available[idx]) { System.out.println("Car is not currently rented!"); }
            else {
                System.out.printf("Returned: %s | Charges: Rs.%d\n",
                    carName[idx], calculateRent(1, pricePerDay[idx]));
                available[idx] = true;
            }
        }

        // MODULE 2 + MODULE 3: Bubble Sort with nested loops
        static void sortCarsByPrice() {
            int[] tId = carId.clone(); String[] tN = carName.clone();
            int[] tP = pricePerDay.clone(); boolean[] tA = available.clone();
            for(int i = 0; i < fleetSize - 1; i++) {
                for(int j = 0; j < fleetSize - i - 1; j++) {
                    if(tP[j] > tP[j+1]) {
                        int tmp=tP[j]; tP[j]=tP[j+1]; tP[j+1]=tmp;
                        int ti=tId[j]; tId[j]=tId[j+1]; tId[j+1]=ti;
                        String ts=tN[j]; tN[j]=tN[j+1]; tN[j+1]=ts;
                        boolean tb=tA[j]; tA[j]=tA[j+1]; tA[j+1]=tb;
                    }
                }
            }
            System.out.printf("\n%-6s %-12s %-12s %s\n","ID","Model","Price/Day","Status");
            System.out.println("-----");
            for(int i = 0; i < fleetSize; i++)
                System.out.printf("%-6d %-12s Rs.%-9d %s\n",
                    tId[i],tN[i],tP[i],tA[i]?"Available":"Rented");
        }

        // MODULE 3: Method with loop
        static void displaySummary() {
            int avail = 0, rented = 0;
            for(int i = 0; i < fleetSize; i++) {
                if(available[i]) avail++; else rented++;
            }
            System.out.println("\n===== FLEET SUMMARY =====");
            System.out.println("Total Cars : " + fleetSize);
            System.out.println("Available Cars: " + avail);
            System.out.println("Rented Cars : " + rented);
            System.out.println("=====");
        }
    }
}
```



OUTPUT SCREENSHOT

```
PS C:\Users\MEGHANA G> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12.8-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\MEGHANA G\AppData\Local\Temp\vscodesws_8b954\jdt_ws\jdt.ls-java-project\bin' 'CarRentalFleetManager'
1. Display Cars      5. Calculate Rent
2. Search Car        6. Sort by Price
3. Rent Car          7. Fleet Summary
4. Return Car        8. Exit
=====
Enter your choice: 4
Enter Car ID to return: 3
Car not found!

===== CAR RENTAL FLEET MANAGER =====
1. Display Cars      5. Calculate Rent
2. Search Car        6. Sort by Price
3. Rent Car          7. Fleet Summary
4. Return Car        8. Exit
=====
```

Sample Output & Execution

Menu Display & Car Listing

```
===== CAR RENTAL FLEET MANAGER =====
1. Display Cars    5. Calculate Rent
2. Search Car     6. Sort by Price
3. Rent Car       7. Fleet Summary
4. Return Car     8. Exit
=====
Enter your choice: 1

ID   Model   Price/Day  Status
-----
101  Swift   Rs.1500   Available
102  Baleno  Rs.1700   Available
103  Creta   Rs.2500   Rented
104  City    Rs.2200   Available
105  Nexon   Rs.1800   Available
```

Rent Operation

```
Enter your choice: 3
Enter Car ID to rent: 102
Rental days: 3
Rented: Baleno | Days: 3 | Total: Rs.5100
```

Fleet Summary After Rental

```
Enter your choice: 7
===== FLEET SUMMARY =====
Total Cars   : 5
Available Cars: 3
Rented Cars  : 2
=====
```

Sort by Price Output

```
Enter your choice: 6

ID   Model   Price/Day  Status
-----
101  Swift   Rs.1500   Available
102  Baleno  Rs.1700   Available
105  Nexon   Rs.1800   Available
104  City    Rs.2200   Available
103  Creta   Rs.2500   Rented
```

✔ All operations execute correctly – rent, return, search, and sort produce accurate console output.

Conclusion & Future Work

Conclusion

The Car Rental Fleet Manager successfully integrates PSP Modules 1, 2, and 3 concepts into a practical, working application – demonstrating computational thinking, control flow, arrays, method overloading, searching, and sorting.

Future Enhancements

- Object-Oriented Programming (Classes, Objects, Inheritance)
- Exception Handling (try-catch-finally, custom exceptions)
- File I/O & Persistent Storage (Database integration)
- Collections Framework (ArrayList, HashMap for dynamic fleet management)
- Functional Programming & Streams (Advanced data processing)
- Graphical User Interface (GUI with Swing/JavaFX)
- Payment Integration & Real-time Tracking

 GitHub Project Link: <https://github.com/gmeghana3105/JAVA-PROJECT/tree/main>

Thank You

Car Rental Fleet Manager - PSPJ Project

GitHub Project Link : <https://github.com/gmeghana3105/JAVA-PROJECT/tree/main>

Presented by: Gundu Meghana

“Programming is not about writing code - it's about solving problems systematically.”